

NCP-AIN AI Fabric Pro Mega Lab Workbook

1. Workbook Model

Every task includes:

- objective
- devices and files
- config actions
- validation
- what good output looks like
- what to look for if broken
- success criteria

GPU node terminology:

- `GPU-A`, `GPU-B`, `GPU-C` are Ubuntu VMs acting like AI worker hosts
- they are not switches
- configure them using the scripts in `configs/hosts/`
- each Ubuntu VM gets both IPv4 and IPv6 on `bond0`

2. Task 1: Import And Base Device Load

Objective:

- get the entire lab booted and loaded cleanly

Devices and files:

- `topology/NCP-AIN\_AI\_FABRIC\_PRO\_MEGA\_LAB.unl`
- `configs/full\_configs/`
- `configs/hosts/`

Config actions:

- import the `.unl`
- boot EOS nodes first
- paste EOS configs in this order:
 - `SPINE1`, `SPINE2`
 - `RR1`, `RR2`
 - `LEAF1`, `LEAF2`, `LEAF3`, `LEAF4`
 - `BL1`, `BL2`
 - `PE1`, `PE2`, `REMOTE-DC`

- boot Ubuntu VMs
- run:
- `sh GPU-A.sh`
- `sh GPU-B.sh`
- `sh GPU-C.sh`
- `sh STORAGE.sh`
- `sh COLLECTOR.sh`

Validation:

- `show running-config | include hostname`
- Linux `ip addr show bond0`
- Linux `ip -6 addr show bond0`
- Linux `cat /proc/net/bonding/bond0`

What good output looks like:

- EOS hostname matches the node
- `bond0` has the expected IP
- both `eth0` and `eth1` are active bond members
- IPv6 address is present on `bond0`

What to look for if broken:

- host IP on `eth0` instead of `bond0`
- one bond slave missing

Success criteria:

- all nodes are loaded and hosts are bonded correctly

3. Task 2: Underlay Bring-Up

Objective:

- establish OSPF and jumbo-capable routed links

Devices and files:

- `SPINE1`, `SPINE2`, `RR1`, `RR2`, `LEAF1-4`, `BL1`, `BL2`

Config actions:

- verify all routed interfaces are `no switchport`
- verify MTU `9214` on underlay links
- verify OSPF on all routed links

Validation:

- `show ip ospf neighbor`
- `show ip route ospf`
- `show interfaces mtu`

- ``show ipv6 interface brief``

What good output looks like:

- spines see all expected neighbors
- leaves see both spines
- fabric loopbacks are present in OSPF routes
- routed links show MTU ``9214``
- IPv6 is enabled on cluster loopbacks and tenant gateways

What to look for if broken:

- missing neighbor suggests IP, mask, state, area, or MTU mismatch
- normal ping works but jumbo ping fails suggests MTU issue

Success criteria:

- underlay is stable end-to-end

4. Task 3: MLAG Peer Domains

Objective:

- validate POD1, POD2, and BORDER MLAG health

Devices and files:

- ``LEAF1/LEAF2`, `LEAF3/LEAF4`, `BL1/BL2``

Config actions:

- verify ``Port-Channel100``
- verify ``Vlan4094``
- verify ``mlag configuration``

Validation:

- ``show mlag``
- ``show port-channel summary``
- ``show interfaces trunk``

What good output looks like:

- MLAG healthy
- peer-link bundled
- VLAN ``4094`` allowed

What to look for if broken:

- suspended members
- missing peer VLAN
- port-channel mismatch

Success criteria:

- all MLAG pairs healthy

5. Task 4: Ubuntu VM Bonding And Access Edge

Objective:

- make the Ubuntu worker and service VMs operational

Devices and files:

- `GPU-A`, `GPU-B`, `GPU-C`, `STORAGE`, `COLLECTOR`
- host scripts in `configs/hosts/`

Config actions:

- run the host scripts
- verify access VLANs on `Ethernet20` and `Ethernet21`

Validation:

- `cat /proc/net/bonding/bond0`
- `ip link show bond0`
- `ip -6 route`
- `show lacp neighbor`

What good output looks like:

- `bond0` in `802.3ad`
- both links up
- switch sees LACP neighbor on both sides
- default IPv6 route points to the SVI anycast IPv6 gateway

What to look for if broken:

- one slave active only
- wrong subnet
- MTU too small on Ubuntu VM

Success criteria:

- all Ubuntu VMs are dual-homed and reachable

6. Task 5: EVPN/VXLAN Overlay

Objective:

- bring up the overlay and VNI mappings

Devices and files:

- `RR1`, `RR2`, `LEAF1-4`, `BL1`, `BL2`

Config actions:

- verify EVPN BGP neighbors
- verify `Vxlan1` mappings
- verify shared Loopback2 within each MLAG pair

Validation:

- ``show bgp evpn summary``
- ``show vxlan vni``
- ``show bgp evpn route-type imet``
- ``show bgp evpn route-type mac-ip``
- ``show ipv6 interface brief``

What good output looks like:

- all VTEPs peer to both route reflectors
- VNIs ``10110``, ``10120``, ``10130``, ``10210`` present
- IMET routes present

What to look for if broken:

- EVPN sessions down
- missing VNI mapping
- MAC-IP routes missing after traffic

Success criteria:

- overlay healthy across pods and border

7. Task 6: Tenant Reachability And GPU Flow Checks

Objective:

- validate same-subnet, routed, and tenant-isolated paths

Devices and files:

- Ubuntu VMs
- ``validation_ping_matrix.sh``

Config actions:

- run standard pings first
- run jumbo pings second

Validation:

- ``sh validation_ping_matrix.sh``
- ``ping -c 3 -M do -s 8972 172.16.110.12``
- ``ping6 -c 3 2001:db8:110::12``
- ``ping6 -c 3 2001:db8:120::50``

What good output looks like:

- same-subnet GPU flows work
- GPU-A reaches STORAGE
- GPU-C stays correct in VLAN210 path
- IPv6 same-subnet and routed traffic also works across the cluster

What to look for if broken:

- same-subnet failure points to host edge or MLAG
- routed failure points to VRF or VNI mapping

Success criteria:

- all intended tenant paths work

8. Task 7: RoCE-Style Traffic And Congestion Drill

Objective:

- simulate AI east-west traffic and observe controlled congestion

Devices and files:

- `GPU-A`, `GPU-B`, `STORAGE`
- `ai\_allreduce\_emulator.sh`
- `congestion\_receive\_shaper.sh`
- `clear\_congestion\_shaper.sh`

Config actions:

- start `iperf3 -s` on `GPU-B` and `STORAGE`
- run `sh ai\_allreduce\_emulator.sh` on `GPU-A`
- apply `sh congestion\_receive\_shaper.sh` on `GPU-B`

Validation:

- `show interfaces counters`
- `iperf3` before/after values
- `tc -s qdisc show dev bond0`

What good output looks like:

- baseline throughput is stable
- shaping reduces throughput measurably
- control-plane stays stable

What to look for if broken:

- full collapse suggests broader path issue
- no difference suggests shaper not applied

Success criteria:

- you can explain congestion symptoms with evidence

9. Task 8: Border eBGP And Remote Services

Objective:

- validate remote route learning and sourced reachability

Devices and files:

- `BL1`, `BL2`, `PE1`, `PE2`, `REMOTE-DC`

Config actions:

- verify eBGP peers
- verify `REMOTE-DC` originates `203.0.113.0/24`

Validation:

- `show ip bgp summary`
- `show ip route 203.0.113.0`
- `ping 203.0.113.10 source 100.64.10.11`
- `ping6 2001:db8:203:113::10`

What good output looks like:

- border sessions up
- remote route present
- sourced ping succeeds
- IPv6 remote service test succeeds if you extend the external edge for dual-stack

What to look for if broken:

- session down means interface, ASN, or neighbor problem
- route missing means advertisement/filter issue

Success criteria:

- remote service path works

10. Task 9: Failure And Reconvergence

Objective:

- prove the fabric survives individual link failures

Devices and files:

- all routed fabric nodes

Config actions:

- shut one leaf-spine link
- retest traffic
- shut one border uplink
- retest traffic

Validation:

- `show ip ospf neighbor`
- `show bgp evpn summary`
- traffic retest

What good output looks like:

- underlay reconverges
- EVPN remains up
- data traffic still works

What to look for if broken:

- EVPN session loss from single underlay failure
- partial host failure after path change

Success criteria:

- single failures do not collapse the fabric

11. Task 10: Broken-State Practice

Objective:

- practice timed troubleshooting with prepared faults

Devices and files:

- ``broken_state_pack/``
- ``troubleshooting/``

Config actions:

- inject one fault
- investigate
- restore using paired restore snippet

Validation:

- ``validation/NCP_AIN_MEGA_LAB_validation_commands.txt``

What good output looks like:

- repaired state matches clean-state behavior

What to look for if broken:

- first broken layer: host edge, MLAG, underlay, EVPN, or eBGP

Success criteria:

- you can diagnose and restore quickly